

L10: Streams II

CS1101S: Programming Methodology

Low Kok Lim

October 25, 2023

Readings

- Textbook [Sec. 3.5](#)

Outline

- Review of Basics
- Wrapping Around
- Memoized Streams
- Advanced Examples

Announcements

- Sumobot Competition 2023 Recap



Champion — pcmasterrace



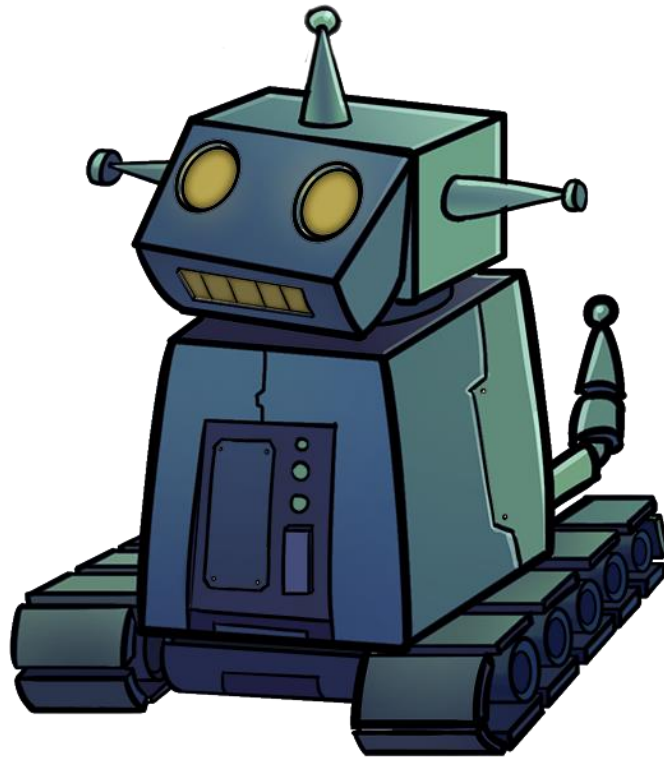
Runner-up — robber robot

Announcements

- **Mastery Check 2 Begins Now**
 - Topics
 - environments of programs
 - memoization
 - streams
- **Reading Assessment 2 (RA2)**
 - 27-Oct-2023, Friday, 10am-12pm
 - Details in Canvas Announcements
- **Practical Assessment (PA)**
 - 4-Nov-2023, Saturday, 1pm-3pm
 - Details in Canvas Announcements

National *Robot* Custodian Appreciation Day

Thank you to all robot custodians!



Outline

- Review of Basics
- Wrapping Around
- Memoized Streams
- Advanced Examples

Idea of Streams

- **Delayed lists**

- Our pairs contain a **data item** as **head** (as usual), but a **function as tail** that can be activated when needed

- **Streams**

- A ***stream of a certain type*** is either **null**, or a **pair** whose **head** is of that type and whose **tail** is a **nullary** function that *returns a stream of that type*

- **Stream discipline**

- Like list discipline, now using streams

Examples — Finite Streams

// An empty stream

```
const s1 = null;
```

// A stream with element 1

```
const s2 = pair(1, () => null);
```

// A stream with elements 1, 2, 3

```
const s3 =  
    pair(1,  
        () => pair(2,  
                    () => pair(3,  
                                () => null))));
```

Example — An Infinite Stream

```
function ones_stream() {  
    return pair(1, ones_stream);  
}
```

```
const ones = ones_stream();
```

```
head(ones); ➔ 1
```

```
head(tail(ones)()); ➔ 1
```

```
head(tail(tail(ones)())()); ➔ 1
```

[Show in
Playground](#)

stream_tail

```
function stream_tail(stream) {  
    return tail(stream());  
}
```

```
const ones = ones_stream();
```

```
head(ones); ➔ 1
```

```
head(stream_tail(ones)); ➔ 1
```

```
head(stream_tail(stream_tail(ones))); ➔ 1
```

[Show in
Playground](#)

Stream Processing

- **Like lists, except**
 - Wrap **tail** in a **function**
 - Use `stream_tail` instead of `tail`
- **Stream support in Source §3**
 - STREAMS provides pre-declared functions for stream processing
 - Examples: `stream_tail`, `stream_map`, `stream_filter`
 - Refer to [STREAMS documentation](#)
 - Refer to [Specification of Source §3](#)

enum_stream

```
function enum_stream(low, hi) {  
    return low > hi  
        ? null  
        : pair(low,  
                () => enum_stream(low + 1, hi));  
}
```

```
const s = enum_stream(1, 100);
```

```
head(s); ➔ 1
```

```
head(stream_tail(s)); ➔ 2
```

```
head(stream_tail(stream_tail(s))); ➔ 3
```

[Show in
Playground](#)

stream_ref

```
function stream_ref(s, n) {  
    return n === 0  
        ? head(s)  
        : stream_ref(stream_tail(s), n - 1);  
}
```

```
const s = enum_stream(1, 100);
```

```
stream_ref(s, 0); ➔ 1
```

```
stream_ref(s, 10); ➔ 11
```

```
stream_ref(s, 99); ➔ 100
```

[Show in
Playground](#)

stream_map and stream_filter

```
function stream_map(f, s) {  
  return is_null(s)  
    ? null  
    : pair(f(head(s)),  
          () => stream_map(f, stream_tail(s)));  
}
```

```
function stream_filter(p, s) {  
  return is_null(s)  
    ? null  
    : p(head(s))  
      ? pair(head(s),  
            () => stream_filter(p, stream_tail(s)))  
      : stream_filter(p, stream_tail(s));  
}
```

[Show in
Playground](#)

Back to the Extreme Example

```
head(stream_tail(stream_filter(  
    is_prime,  
    enum_stream(10000, 1000000)  
))));
```

[Show in
Playground](#)

- **How did it work?**
- Let's see it in action!

Extreme Example (1)

```
head(stream_tail(stream_filter(
  is_prime,
  enum_stream(10000, 1000000))));
```

- Start by calling `enum_stream(10000, 1000000)`, resulting in
`pair(10000, () => enum_stream(10001, 1000000))`

```
function enum_stream(low, hi) {
  return low > hi ? null
    : pair(low,
      () => enum_stream(low + 1, hi));
}
```

Extreme Example (2)

```
head(stream_tail(stream_filter(  
  is_prime,  
  enum_stream(10000, 1000000))));
```

- `is_prime(10000)` is `false`, so `stream_filter` forces evaluation of the tail to produce

```
pair(10001, () => enum_stream(10002, 1000000))
```

```
function stream_filter(p, s) {  
  return is_null(s) ? null  
    : p(head(s))  
      ? pair(head(s),  
        () => stream_filter(p, stream_tail(s)))  
      : stream_filter(p, stream_tail(s));  
}
```

Extreme Example (3)

```
head(stream_tail(stream_filter(
  is_prime,
  enum_stream(10000, 1000000))));
```

- Continue until `is_prime(10007)` is true, and `stream_filter` produces

```
pair(10007,
  () => stream_filter(is_prime,
    pair(10008, () => enum_stream(10009, 1000000))))
```

```
function stream_filter(p, s) {
  return is_null(s) ? null
    : p(head(s))
      ? pair(head(s),
        () => stream_filter(p, stream_tail(s)))
      : stream_filter(p, stream_tail(s));
}
```

Extreme Example (4)

```
head(stream_tail(stream_filter(
  is_prime,
  enum_stream(10000, 1000000))));
```

- Then `stream_tail(pair(10007, () => stream_filter(...)))` forces the tail to evaluate, and `stream_filter` eventually produces
`pair(10009,`
 `() => stream_filter(is_prime,`
 `pair(10010, () => enum_stream(10011, 1000000)))`

```
function stream_filter(p, s) {
  return is_null(s) ? null
    : p(head(s))
      ? pair(head(s),
        () => stream_filter(p, stream_tail(s)))
      : stream_filter(p, stream_tail(s));
}
```

Extreme Example (5)

```
head(stream_tail(stream_filter(  
    is_prime,  
    enum_stream(10000, 1000000))));
```

- Finally `head(pair(10009, () => stream_filter(...)))` produces 10009

More Examples

```
function integers_from(n) {  
    return pair(n, () => integers_from(n + 1));  
}
```

```
const integers = integers_from(1);
```

```
stream_ref(integers, 0); → 1
```

```
stream_ref(integers, 10); → 11
```

```
stream_ref(integers, 99); → 100
```

[Show in
Playground](#)

More Examples

```
function is_divisible(x, y) {  
    return x % y === 0;  
}
```

```
const no_fours =  
    stream_filter(  
        x => !is_divisible(x, 4),  
        integers  
    );
```

```
stream_ref(no_fours, 3); ➔ 5
```

```
stream_ref(no_fours, 100); ➔ 134
```

[Show in
Playground](#)

From Streams to Lists

```
function eval_stream(s, n) {  
    return n === 0  
        ? null  
        : pair(head(s),  
                eval_stream(stream_tail(s), n - 1));  
}
```

```
eval_stream(no_fours, 10);  
// [1, [2, [3, [5, [6, [7, [9, [10, [11, [13,  
// null]]]]]]]]]
```

[Show in
Playground](#)

Fibonacci Numbers

```
function fibgen(a, b) {  
    return pair(a, () => fibgen(b, a + b));  
}  
  
const fibs = fibgen(0, 1);  
  
eval_stream(fibs, 10);  
// [0, [1, [1, [2, [3, [5, [8, [13, [21, [34,  
// null]]]]]]]]]]
```

[Show in
Playground](#)

Replace

- **Wanted:** A function `replace` that creates a new stream by replacing in a given stream a particular value by another value
- **Example:**

```
replace(more_and_more, 1, 0);
```

produces the stream that contains

0, 0, 2, 0, 2, 3, 0, 2, 3, 4, 0, 2, 3, 4, 5, ...

Replace

```
function replace(s, a, b) {  
  return is_null(s)  
    ? null  
    : pair((head(s) == a) ? b : head(s),  
          () => replace(stream_tail(s), a, b));  
}
```

```
const s = replace(more_and_more, 1, 0);
```

```
eval_stream(s, 15);  
// [0, [0, [2, [0, [2, [3, [0, [2, [3, [4,  
// [0, [2, [3, [4, [5, null]]]]]]]]]]]]]]]]]]]]
```

[Show in
Playground](#)

List to Infinite Stream

- **Wanted:** A function `list_to_inf_stream` that takes a list and returns a stream that repeats the list infinitely
- **Example:**

```
list_to_inf_stream(list(1, 2, 3));
```

produces the stream that contains

1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, 2, 3, ...

List to Infinite Stream

```
function list_to_inf_stream(xs) {
  function helper(ys) {
    return is_null(ys)
      ? helper(xs)
      : pair(head(ys), () => helper(tail(ys)));
  }
  return is_null(xs) ? null : helper(xs);
}
```

```
const s = list_to_inf_stream(list(1, 2, 3));
```

```
eval_stream(s, 10);
// [1, [2, [3, [1, [2, [3, [1, [2, [3, [1,
null]]]]]]]]]
```

[Show in
Playground](#)

Why do we need streams?

- Streams allow us to represent **infinite sequences!**
- Streams allow us to use **sequence manipulation** (e.g. map, filter), while keeping the efficiency of **incremental computation**
- Example applications
 - **Video & Audio streaming**
 - Process and play A/V on-the-fly without entire file available
 - **Stream editing**
 - Process a file as a stream (e.g. find and replace text using AWK)

Outline

- Review of Basics
- Wrapping Around
- Memoized Streams
- Advanced Examples

Recall Stream of Ones

// original version

```
function ones_stream() {  
    return pair(1, ones_stream);  
}  
const ones = ones_stream();
```

// alternative version

```
const ones = pair(1, () => ones);
```

[Show in
Playground](#)

Defining a Repeating Sequence

- **Wanted:** Stream containing 1, 2, 3, 1, 2, 3, 1, 2, 3, ...

```
// A stream containing 1, 2, 3, 1, 2, 3, ...
const rep123 =
    pair(1,
        () => pair(2,
            () => pair(3,
                () => rep123))));
```

```
eval_stream(rep123, 15);  
// [1, [2, [3, [1, [2, [3, [1, [2, [3,  
// [1, [2, [3, [1, [2, [3, null]]]]]]]]]]]]]
```

[Show in Playground](#)

Adding Two Streams

- **Wanted:** A function `add_streams` that takes two streams and returns a stream that contains the element-wise sums
- **Example:** Adding

1 2 3 4 5 6 ...

1 1 2 2 3 3 ...

should return

2 3 5 6 8 9 ...

Adding Streams

```
function add_streams(s1, s2) {  
  return is_null(s1)  
    ? s2  
    : is_null(s2)  
    ? s1  
    : pair(head(s1) + head(s2),  
           () => add_streams(stream_tail(s1),  
                              stream_tail(s2)));  
}
```

[Show in
Playground](#)

Fibonacci Numbers, Again

```
const fibs =  
  pair(0,  
    () => pair(1,  
      () => add_streams(  
        stream_tail(fibs),  
        fibs)));  
  
eval_stream(fibs, 10);  
// [0, [1, [1, [2, [3, [5, [8, [13, [21, [34,  
// null]]]]]]]]]]
```

[Show in
Playground](#)

Fibonacci Numbers, Again

```
const fibs =
  pair(0,
    () => pair(1,
      () => add_streams(
        stream_tail(fibs),
        fibs)));
```

		1	1	2	3	5	8	13	...	= stream_tail(fibs)
+		0	1	1	2	3	5	8	...	= fibs

	0	1	1	2	3	5	8	13	21	... = fibs

Fibonacci Numbers, Again

```
const fibs =
  pair(0,
    () => pair(1,
      () => add_streams(
        stream_tail(fibs),
        fibs)));
```

		1	1	2	3	5	8	13	...	= stream_tail(fibs)
+		0	1	1	2	3	5	8	...	= fibs

	0	1	1	2	3	5	8	13	21	... = fibs

Fibonacci Numbers, Again

```
const fibs =
  pair(0,
    () => pair(1,
      () => add_streams(
        stream_tail(fibs),
        fibs)));
```

		1	1	2	3	5	8	13	...	= stream_tail(fibs)
+		0	1	1	2	3	5	8	...	= fibs

	0	1	1	2	3	5	8	13	21	... = fibs

Fibonacci Numbers, Again

```
const fibs =
  pair(0,
    () => pair(1,
      () => add_streams(
        stream_tail(fibs),
        fibs)));
```

		1	1	2	3	5	8	13	...	= stream_tail(fibs)
+		0	1	1	2	3	5	8	...	= fibs

	0	1	1	2	3	5	8	13	21	... = fibs

Integers Revisited

```
const ones = pair(1, () => ones);

const integers =
  pair(1, () => add_streams(ones, integers));

eval_stream(integers, 10);
// [1, [2, [3, [4, [5, [6, [7, [8, [9, [10,
// null]]]]]]]]]]
```

[Show in
Playground](#)

Integers Revisited

```
const ones = pair(1, () => ones);

const integers =
  pair(1, () => add_streams(ones, integers));
```

	1	1	1	1	1	1	...	= ones
+	1	2	3	4	5	6	...	= integers

	1	2	3	4	5	6	7	...
								= integers

Outline

- Review of Basics
- Wrapping Around
- Memoized Streams
- Advanced Examples

Memoization With Lazy Evaluation

```
function memo_fun(fun) {  
  let already_run = false;  
  let result = undefined;  
  
  function mfun() {  
    if (!already_run) {  
      result = fun();  
      already_run = true;  
      return result;  
    } else {  
      return result;  
    }  
  }  
  
  return mfun;  
}
```

[Show in
Playground](#)

Memoized Stream Example 1

```
const onesA = pair(1, () => ms("A", onesA));

function ms(m, s) {
  display(m);
  return s;
}

stream_ref(onesA, 3);
```

Output:

"A"

"A"

"A"

1

[Show in
Playground](#)

Memoized Stream Example 1

```
const onesB =  
  pair(1, memo_fun(() => ms("B", onesB)));  
  
function ms(m, s) {  
  display(m);  
  return s;  
}  
  
stream_ref(onesB, 3);
```

Output:

"B"

1

[Show in
Playground](#)

Memoized Stream Example 1

```
const onesB =  
  pair(1, memo_fun(() => ms("B", onesB)));
```

```
stream_ref(onesB, 3);
```

```
→ stream_ref(display("B"); memoize onesB; return onesB, 2)  
→ stream_ref(return memoized onesB, 1)  
→ stream_ref(return memoized onesB, 0)  
→ head(onesB)  
→ 1
```

```
function stream_ref(s, n) {  
  return n === 0  
    ? head(s)  
    : stream_ref(stream_tail(s), n - 1);  
}
```


Memoized Stream Example 2

```
function m_integers_from(n) {  
  return pair(n,  
    memo_fun(() => ms("M: " + stringify(n),  
      m_integers_from(n + 1))));  
}
```

```
const m_integers = m_integers_from(1);
```

```
stream_ref(m_integers, 0);
```

1

```
stream_ref(m_integers, 1);
```

"M: 1"
2

```
stream_ref(m_integers, 2);
```

"M: 2"
3

[Show in
Playground](#)

Memoized Stream Example 2

```
stream_ref(m_integers, 5);
```

Output:

"M: 3"

"M: 4"

"M: 5"

6

```
stream_ref(m_integers, 5);
```

Output:

6

[Show in
Playground](#)

Why Don't We Always Do This?

- **Nature of functions**

- Functions often have an ***effect***, beyond the value they are computing

- **Side effects**

- When we are talking about lazy evaluation, these effects are called ***side effects***

- **Pure functional programming**

- Programming without side-effects
- All values can be memoized
 - Results not affected by memoization

Outline

- Review of Basics
- Wrapping Around
- Memoized Streams
- **Advanced Examples**

Example — Generating Primes

[Show in
Playground](#)

```
const primes =  
  pair(2,  
    () => stream_filter(is_prime,  
                        integers_from(3)));  
  
eval_stream(primes, 10);  
// [2, [3, [5, [7, [11, [13, [17, [19, [23, [29,  
// null]]]]]]]]]
```

- Can we make a prime stream without using `is_prime`?

Sieve of Eratosthenes

Start with integers beginning with 2

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 ...

Filter away multiples of 2 that are greater than 2

2 3 5 7 9 11 13 15 17 ...

Next left over integer is 3

2 **3** 5 7 9 11 13 15 17 ...

Filter away multiples of 3 that are greater than 3

2 **3** 5 7 11 13 17 ...

Next left over integer is 5

2 3 **5** 7 11 13 17 ...

And, so on...

Sieve of Eratosthenes

```
function is_divisible(x, y) {  
    return x % y === 0;  
}  
  
function sieve(s) {  
    return pair(head(s),  
        () => sieve(stream_filter(  
            x => !is_divisible(x, head(s)),  
            stream_tail(s))));  
}  
  
const primes = sieve(integers_from(2));  
  
eval_stream(primes, 10);  
// [2, [3, [5, [7, [11, [13, [17, [19, [23, [29,  
// null]]]]]]]]]]
```

[Show in
Playground](#)

Example — Square Roots by Newton's Method

```
function improve(guess, x) {  
    return average(guess, x / guess);  
}  
  
function sqrt_iter(guess, x) {  
    if (good_enough(guess, x)) {  
        return guess;  
    } else {  
        return sqrt_iter(improve(guess, x), x);  
    }  
}  
  
function sqrt(x) {  
    return sqrt_iter(1.0, x);  
}
```


Using Streams for Iteration

```
function average(a, b) {  
    return (a + b) / 2;  
}  
function improve(guess, x) {  
    return average(guess, x / guess);  
}  
  
function sqrt_stream(x) {  
    const guesses = pair(1.0,  
        () => stream_map(  
            guess => improve(guess, x),  
            guesses));  
    return guesses;  
}  
  
eval_stream(sqrt_stream(2), 6);
```

[Show in
Playground](#)

Using Streams for Iteration

```
eval_stream(sqrt_stream(2), 6);  
  
// [1,  
// [1.5,  
// [1.41666666666666665,  
// [1.4142156862745097,  
// [1.4142135623746899,  
// [1.414213562373095,  
// null]]]]]
```

[Show in
Playground](#)

Example — Approximating π

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$$

- Function to produce a stream of ***summands*** (i.e. 1, -1/3, 1/5, ...)

```
function pi_summands(n) {  
    return pair(1 / n,  
        () => stream_map(x => -x,  
            pi_summands(n + 2)));  
}
```

Approximating π

```
const pi_stream =  
    scale_stream(partial_sums(pi_summands(1)), 4);  
  
// Returns a stream whose elements are  
// s0, s0 + s1, s0 + s1 + s2, ...  
function partial_sums(s) {  
    // Bring your solutions to Reflection class  
}  
  
function scale_stream(s, f) {  
    return stream_map(x => x * f, s);  
}
```

[Show in
Playground](#)

Approximating π

```
const pi_stream =  
    scale_stream(partial_sums(pi_summands(1)), 4);  
  
eval_stream(pi_stream, 8);  
  
// [4,  
// [2.6666666666666667,  
// [3.4666666666666667,  
// [2.8952380952380956,  
// [3.3396825396825403,  
// [2.9760461760461765,  
// [3.2837384837384844,  
// [3.017071817071818,  
// null]]]]]]]
```

[Show in
Playground](#)

Summary

- In streams, functions serve as **pickles**
 - Open the jar (apply the function) only when you need them
- Streams can represent “**infinite**” **data structures**
- **Laziness** of streams avoids problem of **non-termination**
- **Memoization** can be used on streams
- Streams can be used for **iterations**